PEMROGRAMAN BERORIENTASI OBJEK 2026

Pengenalan PBO dan Persiapan Alat

Cara pandang baru dalam menyusun program, alasan kita membutuhkannya, dan alat untuk praktik

PERTEMUAN

01

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan arti **paradigma pemrograman** dan membedakan paradigma **prosedural** dengan **berorientasi objek**.
2. Menjelaskan masalah program prosedural saat program membesar, dengan contoh data menu KedaiKita yang tertukar.
3. Memandang sistem sebagai kumpulan **objek** yang punya **atribut** dan ***method***, serta saling mengirim pesan.
4. Menyebutkan **empat pilar PBO** beserta analogi sederhananya.
5. Menimbang kelebihan PBO dan mengenali situasi ketika PBO kurang cocok.
6. Memilih jalur praktik (Python atau Java), memeriksa alatnya, dan menjalankan program pertamamu.

Agenda Hari Ini

1. Kasir KedaiKita yang kacau
2. Paradigma prosedural dan masalahnya
3. Dunia penuh objek
4. Empat pilar dan kapan memakai PBO
5. Dua jalur dan persiapan alat
6. Studi kasus KedaiKita

BAGIAN 1

1

Kasir KedaiKita yang Kacau

Kejadian kecil yang membuka pintu ke PBO.

Suatu Sore di KedaiKita

- KedaiKita, kafe kecil di Jl. Pramuka No. 10, Samarinda, menyimpan data menu di **tiga daftar terpisah**: nama, harga, dan stok.
- Menu pertama ada di urutan pertama pada ketiga daftar, menu kedua di urutan kedua, dan seterusnya.
- Pemilik kedai ingin menu tampil **urut abjad**, lalu menambahkan satu perintah untuk mengurutkan **daftar nama**.
- Hasilnya, Es Teh Manis menjadi **Rp18.000**, sedangkan Nasi Goreng Kampung hanya **Rp8.000**.

Perhatikan

Tidak ada pesan *error* sama sekali. Program berjalan “lancar”, tetapi datanya **tertukar**.

Tiga Paradigma Pemrograman

Paradigma	Ide utama	Analogi dapur	Contoh bahasa
Prosedural (*procedural*)	Urutan langkah dan fungsi yang mengolah data	Satu buku resep, semua bahan di meja bersama	C, Pascal
Berorientasi objek (*object-oriented*)	Kumpulan objek yang membawa data dan perilakunya sendiri, lalu bekerja sama	Tim petugas dengan meja dan tugas masing-masing	Java, C#, Python
Fungsional (*functional*)	Rangkaian fungsi yang mengubah masukan menjadi keluaran	Jalur produksi: bahan masuk, diolah tahap demi tahap, hasil keluar	Haskell

Paradigma pemrograman: cara pandang atau gaya dalam menyusun program, yaitu bagaimana masalah dipecah dan di mana data disimpan.

Masalah Gaya Prosedural saat Program Membesar

- **Data dan fungsi terpisah.** Data satu menu tersebar di tiga daftar, direkatkan hanya oleh **indeks** yang sama (**data paralel**).
- **Data global bisa diubah siapa saja.** Saat data rusak, semua fungsi harus diperiksa untuk mencari “pelakunya”.
- **Mudah rusak saat diubah.** Satu perintah yang kurang lengkap, atau satu fungsi yang terlewat, membuat data tertukar.
- Untuk program kecil, gaya prosedural **baik-baik saja**: mudah ditulis dan mudah dipahami.

Penting

Inti masalahnya: **data satu benda tercerai-berai** dan **semua fungsi bebas mengubah semua data**.

BAGIAN 2

2

Dunia Penuh Objek

Cara pandang yang menyatukan data dan perilaku.

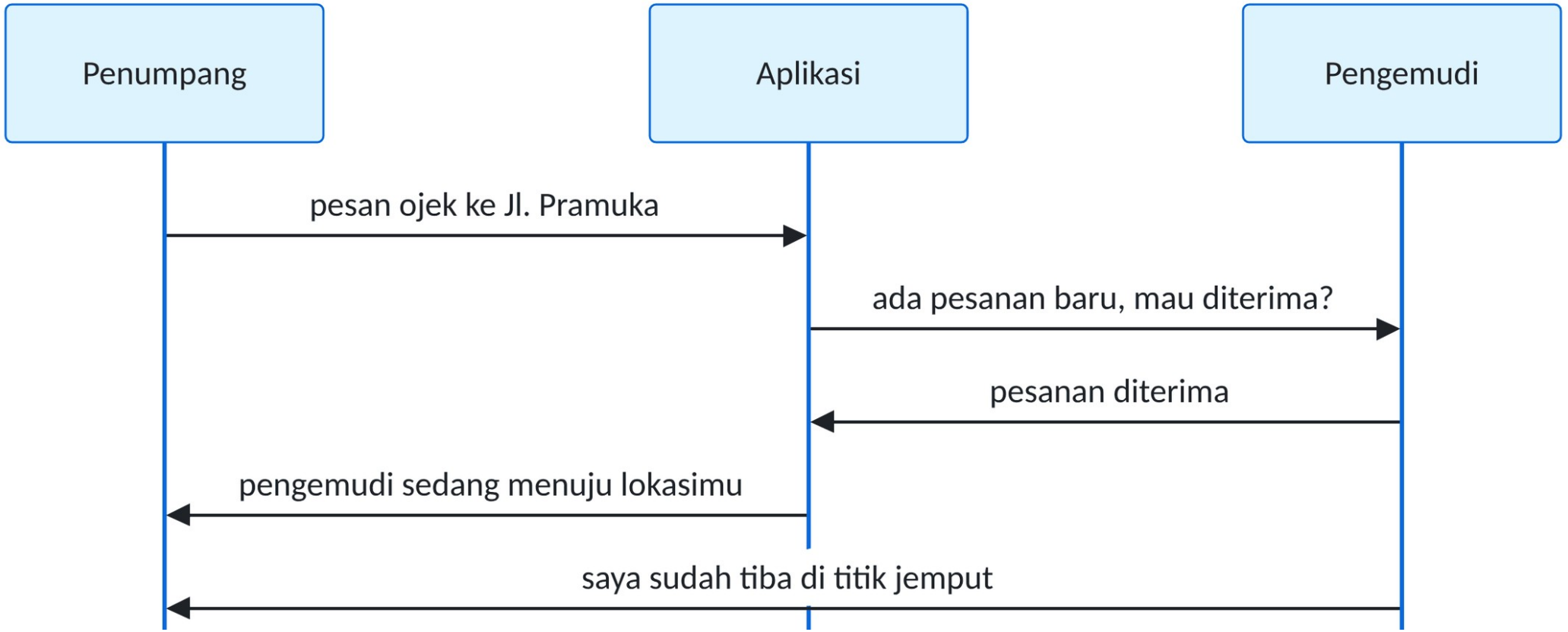
PBO: Program sebagai Kumpulan Objek

Pemrograman Berorientasi Objek (PBO)

Paradigma yang menyusun program sebagai kumpulan **objek**. Setiap objek menyatukan **data** (disebut **atribut**) dan **perilaku** (disebut ***method***). Objek-objek bekerja sama dengan **saling mengirim pesan**.

- Dalam bahasa Inggris: *Object-Oriented Programming* (OOP).
- Di kampus, objek mahasiswa membawa nama, NIM, dan IPK, serta bisa mengisi KRS dan membayar UKT.
- Di kafe, setiap menu adalah **satu objek** yang membawa nama, harga, dan stoknya sendiri.

Objek Saling Mengirim Pesan



Mengirim pesan ke objek berarti meminta objek itu melakukan sesuatu. Di dalam kode, wujudnya adalah pemanggilan ***method***.

Prosedural dan Berorientasi Objek: Apa Bedanya?

Aspek	Prosedural	Berorientasi objek
Unit utama program	fungsi (langkah-langkah)	objek (data + perilaku)
Letak data satu benda	tersebar di beberapa variabel atau daftar	terkumpul di dalam satu objek
Siapa yang mengubah data	fungsi mana saja	terutama *method* milik objek itu sendiri
Cara bagian program bekerja sama	fungsi memanggil fungsi lain sambil mengoper data	objek mengirim pesan ke objek lain
Paling terasa manfaatnya untuk	program kecil dan skrip sekali pakai	program besar yang terus berkembang

Satu Rancangan, Ribuan Ponsel

Analogi

- Insinyur membuat **satu rancangan** model ponsel: data yang dimiliki (warna, sisa baterai) dan kemampuannya.
- Dari rancangan itu, pabrik memproduksi **ribuan ponsel** sungguhan dengan nilai data yang bisa berbeda.
- Ponsel Ayu berwarna hitam dengan sisa baterai 80%, sedangkan ponsel Bima biru dengan sisa baterai 35%.

Dunia pabrik ponsel	Istilah PBO
Rancangan model ponsel	**Kelas** (*class*)
Ponsel milik Ayu, ponsel milik Bima	**Objek** (*object*)
Warna, sisa baterai	**Atribut**
Menelepon, mengirim pesan, mengisi daya	***Method***

BAGIAN 3

3

Empat Pilar dan Kapan Memakai PBO

Cukup kenali nama dan gambarannya; setiap pilar punya pertemuan khusus.

Empat Pilar PBO Sekilas

Pilar	Inti dalam satu kalimat	Analogi	Pertemuan
Enkapsulasi	Data dibungkus dan hanya diubah lewat pintu resmi	Mesin ATM	04
Pewarisan	Kelas baru mewarisi ciri kelas yang sudah ada	Kendaraan bermotor → motor dan mobil	07
Polimorfisme	Satu perintah, cara kerjanya berbeda-beda	Tombol “Bayar”	09
Abstraksi	Tampilkan yang penting, sembunyikan yang rumit	Menyetir mobil	10

Sebagian buku hanya menyebut tiga pilar karena abstraksi dianggap dasar dari semuanya. Mata kuliah ini memakai empat pilar.

Kapan PBO Bermanfaat dan Kapan Kurang Cocok

Kelebihan PBO

- Data tidak tercerai-berai: harga tidak lagi tertukar saat menu diurutkan.
- Lebih mudah dirawat: aturan stok cukup diubah di kelas menu.
- Bisa dipakai ulang (*reusable*), bahkan di program lain.
- Mudah dikerjakan bersama: satu orang mengurus pembayaran, yang lain laporan.
- Dekat dengan dunia nyata dan data lebih terlindungi (enkapsulasi).

Kapan PBO kurang cocok

- Program sangat kecil atau skrip sekali pakai, misalnya mengganti nama 200 *file* foto.
- Pengolahan data berupa aliran langkah: membaca, menyaring, lalu menjumlahkan.
- Rancangan berlebihan (*over-engineering*): terlalu banyak kelas untuk masalah sederhana.
- Dalam praktik, paradigma sering **dicampur**: di dalam *method* tetap ada langkah prosedural.

BAGIAN 4

4

Dua Jalur dan Persiapan Alat

Konsep yang sama, dua cara menulis. Pilih satu jalur utama: Python atau Java.

Alat dan Cek Instalasi

Kebutuhan	Jalur Python	Jalur Java
Bahasa	Python 3.10 atau lebih baru	JDK (*Java Development Kit*) 17 atau lebih baru
Editor	VS Code + ekstensi Python	VS Code + Extension Pack for Java
Cek di terminal	`python --version` (macOS/Linux: `python3 --version`)	`java -version` lalu `javac -version`
Hasil yang benar	nomor versi, misalnya `Python 3.12.4`	nomor versi 17 atau lebih, misalnya 17, 21, atau 25

Ketik perintahnya di **terminal**, bukan di dalam *file*. Panduan pemasangan lengkap ada di folder 00-Persiapan.

Tiga Tempat Mengetik

1 Editor (VS Code)

halo.py

```
1 print("Halo!")
2
```

Untuk: menulis program.

Isinya disimpan sebagai *file*, misalnya halo.py atau Halo.java. Simpan dengan Ctrl+S.

2 Terminal

```
PS C:\pbo> python halo.py
Halo!
PS C:\pbo> java Halo.java
Halo!
```

Untuk: memberi perintah.

Pindah folder (cd), menjalankan program (python, javac, java). Tandanya: PS C:\...> atau $

3 Mode interaktif

```
>>> 18000 * 2
36000
jshell> 18000 * 2
$1 ==> 36000
```

Untuk: mencoba satu baris.

Python: ketik python saja. Tandanya >>>, keluar: exit(). Java: ketik jshell. Tandanya jshell>, keluar: /exit.

Perintah seperti `python --version` diketik di **terminal**, bukan di editor dan bukan setelah tanda >>> atau `jshell>`.

Menjalankan Program Pertama

Jalankan dengan tombol **Run** di VS Code, atau di terminal: `python contoh01_halo.py` dan `java Contoh01Halo.java`.

Python python/contoh/contoh01_halo.py

```python
def main():
    print("=== PROGRAM PERTAMAKU ===")
    print("Halo, Samarinda!")
    # ... (dilewati)


if __name__ == "__main__":
    main()
```

Java java/contoh/Contoh01Halo.java

```java
public class Contoh01Halo {

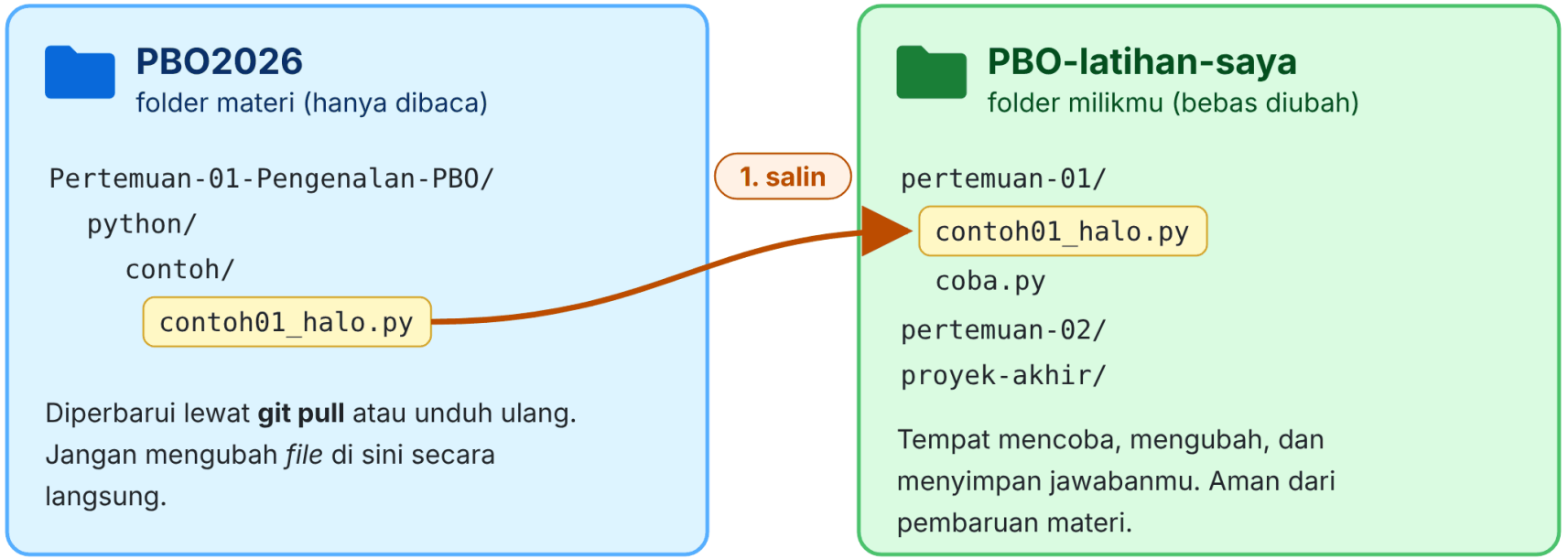
    public static void main(String[] args) {
        System.out.println("=== PROGRAM PERTAMAKU ===");
        System.out.println("Halo, Samarinda!");
        // ... (dilewati)
    }
}
```

Output

```
=== PROGRAM PERTAMAKU ===
Halo, Samarinda!
...
```

Simpan Latihan di Folder Sendiri

Contoh susunan di dalam folder Documents



1. Salin *file* yang ingin dicoba. **2.** Buka salinannya, ubah, lalu jalankan.

Hasilnya: materi asli tetap utuh, dan pembaruan materi tidak bentrok dengan pekerjaanmu.

Folder materi hanya dibaca. Salin *file* contoh ke `PBO-latihan-saya/pertemuan-01/`, lalu ubah dan jalankan di sana.

STUDI KASUS



Studi Kasus KedaiKita

Dua program dengan skenario sama, berbeda cara menyimpan data.

Versi Prosedural: Hanya Nama yang Diurutkan

Studi kasus KedaiKita

Data menu ada di tiga *list/array* paralel. Yang diurutkan hanya daftar nama; harga dan stok tetap di tempatnya.

Python python/studi-kasus/kedaikita_prosedural.py

```python
# ... nama_menu berisi tiga nama menu
harga_menu = [18000, 25000, 8000]
stok_menu = [20, 15, 30]
# ... (dilewati)
pesanan_budi = [(0, 2), (2, 1)]
total = hitung_total(pesanan_budi)
# ... (dilewati)
print("Mengurutkan nama menu sesuai abjad...")
nama_menu.sort()
tampilkan_menu()
```

Java java/studi-kasus/KedaiKitaProsedural.java

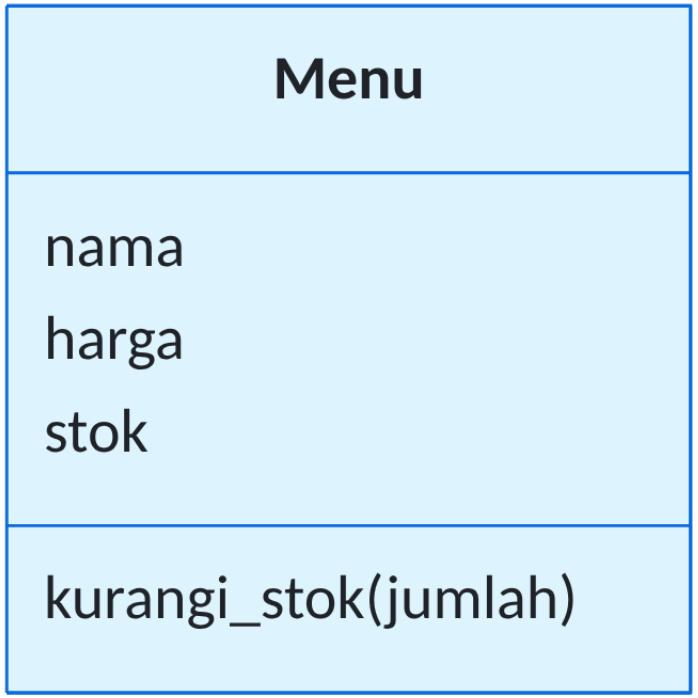
```java
// ... namaMenu berisi tiga nama menu
static int[] hargaMenu = {18000, 25000, 8000};
static int[] stokMenu = {20, 15, 30};
// ... (dilewati)
int[][] pesananBudi = {{0, 2}, {2, 1}};
int total = hitungTotal(pesananBudi);
// ... (dilewati)
System.out.println("Mengurutkan nama menu sesuai abjad...");
Arrays.sort(namaMenu);
tampilkanMenu();
```

Output

```
Mengurutkan nama menu sesuai abjad...
=== DAFTAR MENU KEDAIKITA ===
1. Es Teh Manis - Rp18000 (stok 18)
2. Kopi Susu Gula Aren - Rp25000 (stok 15)
3. Nasi Goreng Kampung - Rp8000 (stok 29)
```

Rancangan Kelas Menu pada Versi Objek

Studi kasus KedaiKita



Setiap menu menjadi **satu objek** yang membawa nama, harga, dan stoknya sendiri, serta bisa mengurangi stoknya.

Versi Objek: Menu Pindah Utuh

Studi kasus KedaiKita

Tiga objek Menu disimpan dalam satu daftar. Saat diurutkan, yang berpindah adalah **objeknya**.

Python python/studi-kasus/kedaikita_oop.py

```python
kopi = Menu("Kopi Susu Gula Aren", 18000, 20)
nasi_goreng = Menu("Nasi Goreng Kampung", 25000, 15)
es_teh = Menu("Es Teh Manis", 8000, 30)
daftar_menu = [kopi, nasi_goreng, es_teh]
# ... (dilewati)
kopi.kurangi_stok(2)
es_teh.kurangi_stok(1)
# ... (dilewati)
daftar_menu.sort(key=lambda menu: menu.nama)
tampilkan_menu(daftar_menu)
```

Java java/studi-kasus/KedaiKitaOop.java

```java
Menu kopi = new Menu("Kopi Susu Gula Aren", 18000, 20);
Menu nasiGoreng = new Menu("Nasi Goreng Kampung", 25000, 15);
Menu esTeh = new Menu("Es Teh Manis", 8000, 30);
ArrayList<Menu> daftarMenu = new ArrayList<>();
// ... daftarMenu diisi kopi, nasiGoreng, esTeh
kopi.kurangiStok(2);
esTeh.kurangiStok(1);
// ... (dilewati)
daftarMenu.sort((a, b) -> a.nama.compareTo(b.nama));
tampilkanMenu(daftarMenu);
```

Output

```
=== DAFTAR MENU KEDAIKITA ===
1. Es Teh Manis - Rp8000 (stok 29)
2. Kopi Susu Gula Aren - Rp18000 (stok 18)
3. Nasi Goreng Kampung - Rp25000 (stok 15)
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“PBO itu sekadar menulis kata `class`.”	PBO adalah **cara berpikir**: program dipandang sebagai objek yang membawa data dan perilakunya sendiri. Program Java hari ini pun memakai `class`, tetapi gayanya masih prosedural.
“Program prosedural itu salah dan kuno.”	Untuk program kecil, gaya prosedural justru sering menjadi pilihan terbaik. Bahkan di dalam *method*, kita tetap menulis langkah-langkah prosedural.
“Kalau program tidak menampilkan *error*, berarti hasilnya benar.”	Belum tentu. KedaiKita versi prosedural berjalan tanpa *error*, tetapi harganya tertukar. Kesalahan logika seperti ini paling berbahaya karena terjadi diam-diam.

Kuis Cepat

Pada KedaiKita versi prosedural, mengapa harga menu **tertukar** setelah daftar menu diurutkan?

A. Python dan Java keliru saat menghitung harga setelah daftar diurutkan

B. Harga disimpan sebagai bilangan pecahan sehingga nilainya menjadi tidak tepat

C. Program menampilkan pesan *error*, lalu berhenti sebelum sempat menampilkan menu

D. Hanya daftar nama yang diurutkan, sedangkan daftar harga dan stok tetap di tempatnya

Kuis Cepat: Jawaban

Pada KedaiKita versi prosedural, mengapa harga menu **tertukar** setelah daftar menu diurutkan?

A. Python dan Java keliru saat menghitung harga setelah daftar diurutkan

B. Harga disimpan sebagai bilangan pecahan sehingga nilainya menjadi tidak tepat

C. Program menampilkan pesan *error*, lalu berhenti sebelum sempat menampilkan menu

✓ Hanya daftar nama yang diurutkan, sedangkan daftar harga dan stok tetap di tempatnya

Jawaban: D

Data satu menu tersebar di tiga daftar yang hanya direkatkan oleh indeks yang sama. Pengurutan hanya memindahkan isi daftar nama sehingga nama di indeks 0 berpasangan dengan harga milik menu lain. Bahasanya tidak salah hitung (A), harganya bilangan bulat (B), dan justru tidak ada pesan *error* sama sekali (C).

Kuis Cepat

Di mesin kasir, tombol “Bayar” bisa memproses pembayaran tunai, QRIS, atau kartu, dan masing-masing bekerja dengan caranya sendiri. Pilar PBO yang **paling** menggambarkan situasi ini adalah …

A Enkapsulasi

B Pewarisan

C Abstraksi

D Polimorfisme

Kuis Cepat: Jawaban

Di mesin kasir, tombol “Bayar” bisa memproses pembayaran tunai, QRIS, atau kartu, dan masing-masing bekerja dengan caranya sendiri. Pilar PBO yang **paling** menggambarkan situasi ini adalah …

A Enkapsulasi

B Pewarisan

C Abstraksi

✓ Polimorfisme

Jawaban: D

Polimorfisme berarti “banyak bentuk”: perintah yang sama (“bayar”) ditanggapi dengan cara berbeda oleh objek yang berbeda. Enkapsulasi (A) tentang membungkus dan melindungi data; pewarisan (B) tentang mewarisi ciri kelas lama. Abstraksi (C) memang ikut berperan, tetapi inti situasinya adalah **satu perintah dengan banyak cara**.

Ringkasan

- **Paradigma pemrograman** adalah cara pandang dalam menyusun program: prosedural, berorientasi objek, dan fungsional.
- Gaya **prosedural** memisahkan data dan fungsi. Pada program besar, data satu benda mudah tercerai-berai.
- KedaiKita versi prosedural membuktikannya: data **tertukar** saat diurutkan, tanpa satu pun pesan *error*.
- **PBO** menyusun program sebagai **objek** yang menyatukan **atribut** dan ***method***, lalu saling **mengirim pesan**.
- **Kelas** adalah rancangan, sedangkan **objek** adalah wujud nyatanya. Detailnya dibahas di Pertemuan 02.
- **Empat pilar PBO**: enkapsulasi, pewarisan, polimorfisme, dan abstraksi, masing-masing punya pertemuan sendiri.
- PBO unggul untuk program besar, tetapi bisa berlebihan untuk skrip kecil. Pilih satu jalur: **Python** atau **Java**.

Langkah Berikutnya

- Pilih jalurmu, cek alat di terminal, lalu jalankan contoh 01—06 dan kedua program KedaiKita.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal (target minimal 8 benar).
- Simpan semua percobaanmu di folder `PBO-latihan-saya/pertemuan-01/`.

Buka bahan pertemuan ini di GitHub:

- **Konsep**
- **Praktik Python**
- **Praktik Java**
- **Jembatan Python ↔ Java**
- **Latihan**
- **Kuis 10 soal**

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-01-Pengenalan-PBO